

YZU · CS351 – AI-ASSISTED SOFTWARE DEVELOPMENT

Final Demo

Two Sum → CSV Mini Database

PRESENTER

Alvin

STUDENT ID

s1123530

COURSE

11402 CS351

AGENDA

Demo Roadmap

01 Personal Website

02 Course Repository

03 Project 0 — Two Sum

04 Project B — CSV Mini Database

05 AI Workflow & Reflection

EVIDENCE-BASED DEMO

Every claim on the following slides is backed by a live screenshot from GitHub — source, tests, commits, pull requests, and green CI runs.

src

tests

CI

pull requests

Personal Website

- Hosted on GitHub Pages
- Hand-written HTML & CSS
- Responsive layout
- Personal branding

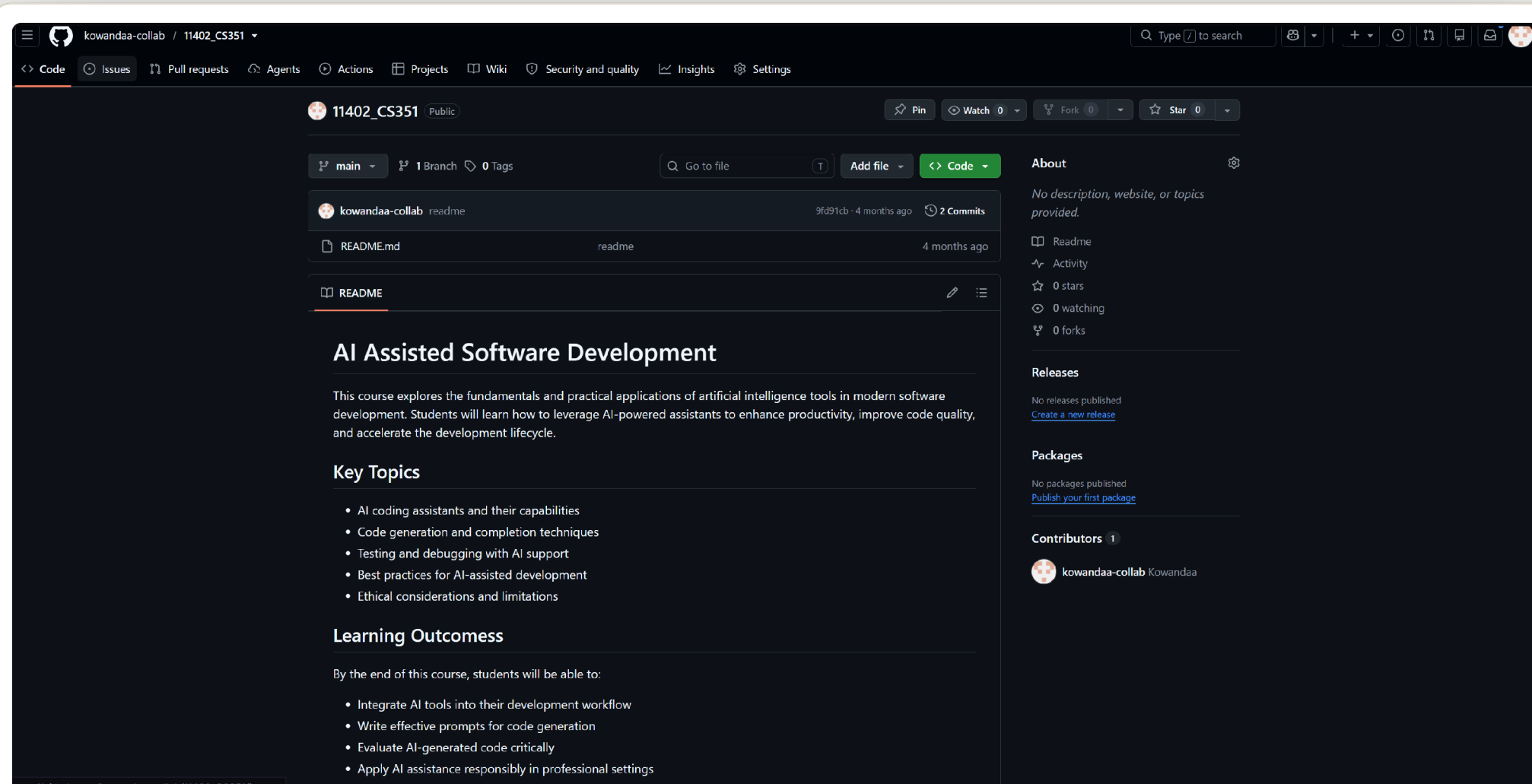
LIVE SITE

Open my website ↗

kowandaa-collab.github.io

click to open in a new tab

Course Repository



— README-driven overview

— Key topics documented

— Clear learning outcomes

— Public on GitHub

Two Sum — The Problem

“Given an array of integers and a target sum, find the indices of the two distinct numbers that add up to the target. Exactly one valid answer exists, and you cannot reuse the same element twice.”

EXAMPLE

```
nums = [2, 7, 11, 15]
target = 9
→ returns [0, 1]
```

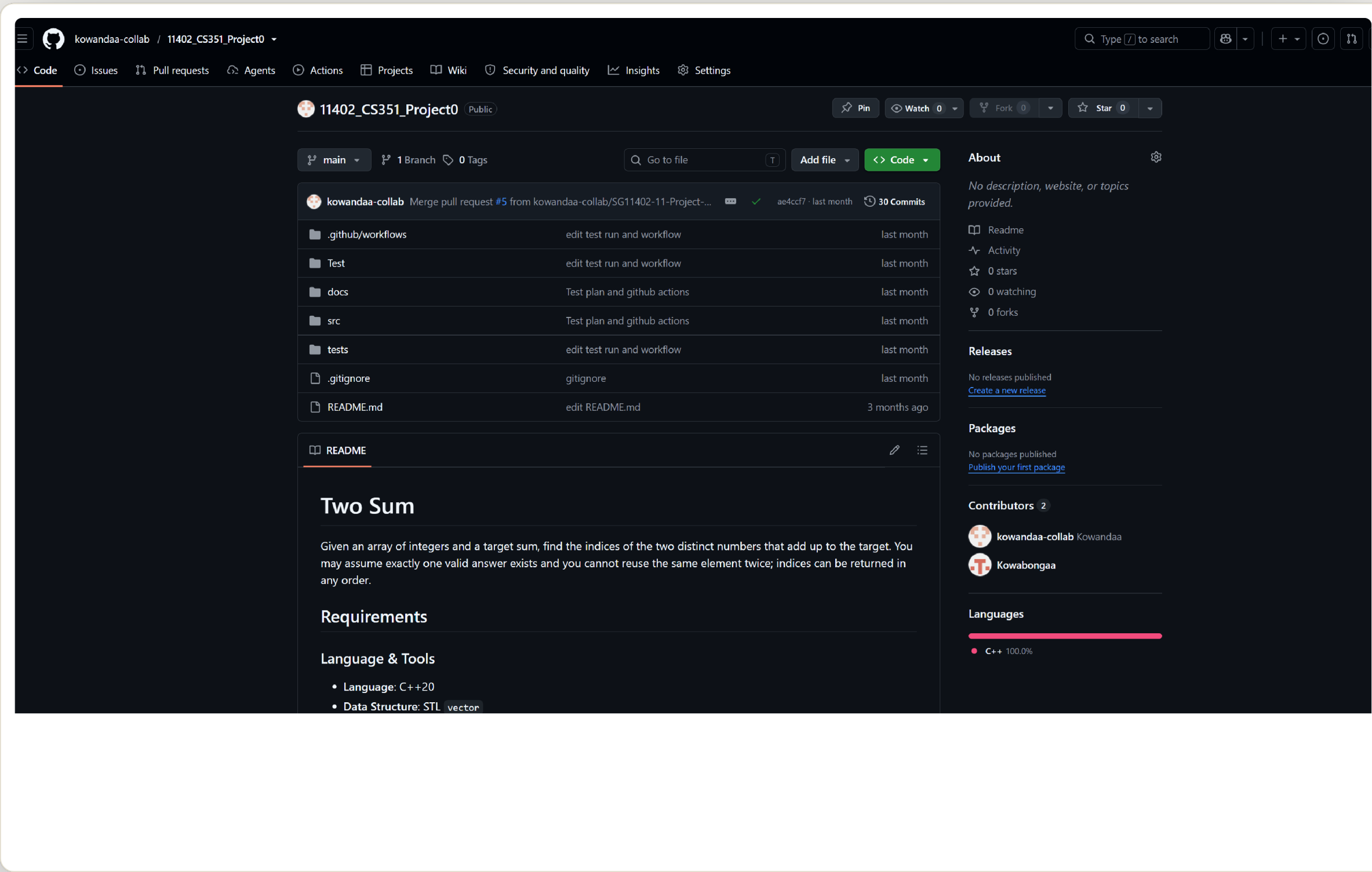
Constraints that drive the design:

- two *distinct* indices
- no element reused
- indices in any order

Repository Structure

C++20

30 commits



LAYOUT

src/ twosum.cpp

tests/ test cases

docs/ dev notes

.github/ CI workflow

README.md

Code, tests, docs and CI — cleanly separated.

Algorithm & Implementation

TwoSumArray

Brute force — nested loops. My trusted baseline.

$O(n^2)$

TwoSumHashTable

Single pass with unordered_map. Insert *after* lookup — never reuses an element.

$O(n)$

The required `twoSum()` wraps the hash-table version.

11402_CS351_Project0 / src / twosum.cpp

Kowabongaa Test plan and github actions ✓

Code Blame 42 lines (36 loc) • 1.21 KB

```
1  #include <algorithm>
2  #include <unordered_map>
3  #include <vector>
4
5  std::vector<int> TwoSumArray(const std::vector<int>& nums, int target) {
6      for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
7          for (int j = i + 1; j < static_cast<int>(nums.size()); ++j) {
8              if (nums[i] + nums[j] == target) {
9                  return {i, j}; // i < j by construction
10             }
11         }
12     }
13     return {};
14 }
15
16 std::vector<int> TwoSumHashTable(const std::vector<int>& nums, int target) {
17     std::unordered_map<int, int> seen; // value -> index
18     seen.reserve(nums.size());
19
20     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
21         const int need = target - nums[i];
22         const auto it = seen.find(need);
23         if (it != seen.end()) {
24             int a = it->second;
25             int b = i;
26             if (a > b) std::swap(a, b);
27             return {a, b};
28         }
29
30         // Insert AFTER lookup so we never reuse the same element twice
31         if (seen.find(nums[i]) == seen.end()) {
32             seen.emplace(nums[i], i);
33         }
34     }
35
36     return {};
37 }
38
39 // Required signature from README.md
40 std::vector<int> twoSum(const std::vector<int>& nums, int target) {
41     return TwoSumHashTable(nums, target);
42 }
```

Testing Strategy

Each case runs `runOne()` — all three versions cross-checked.

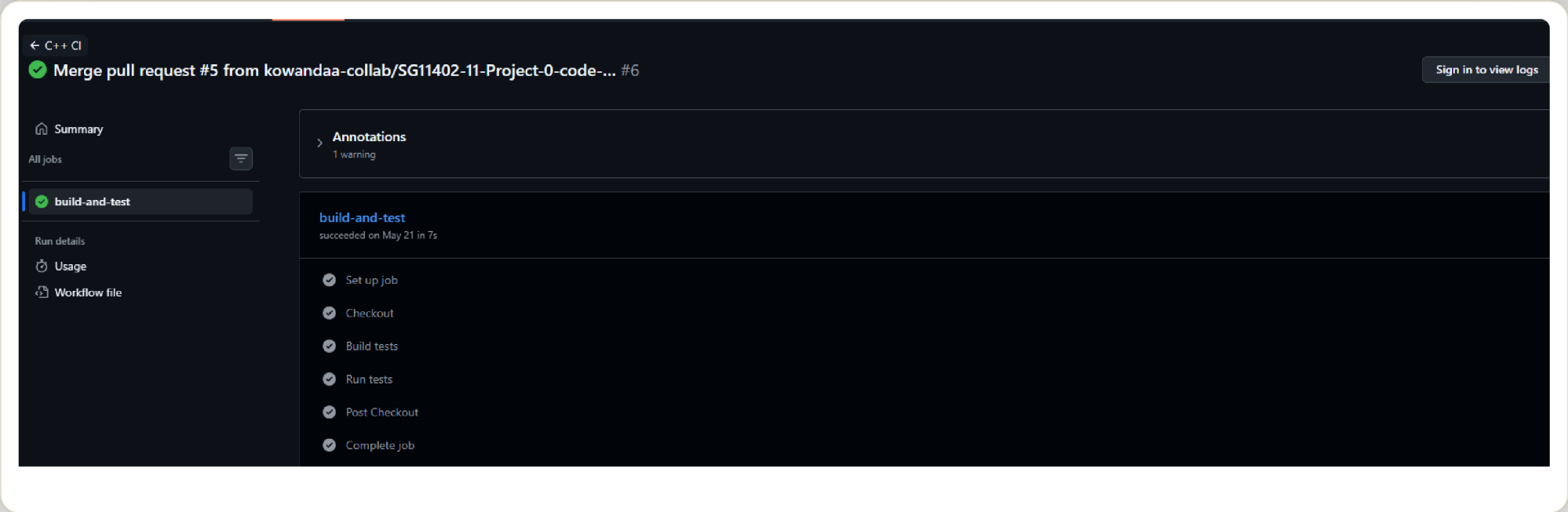
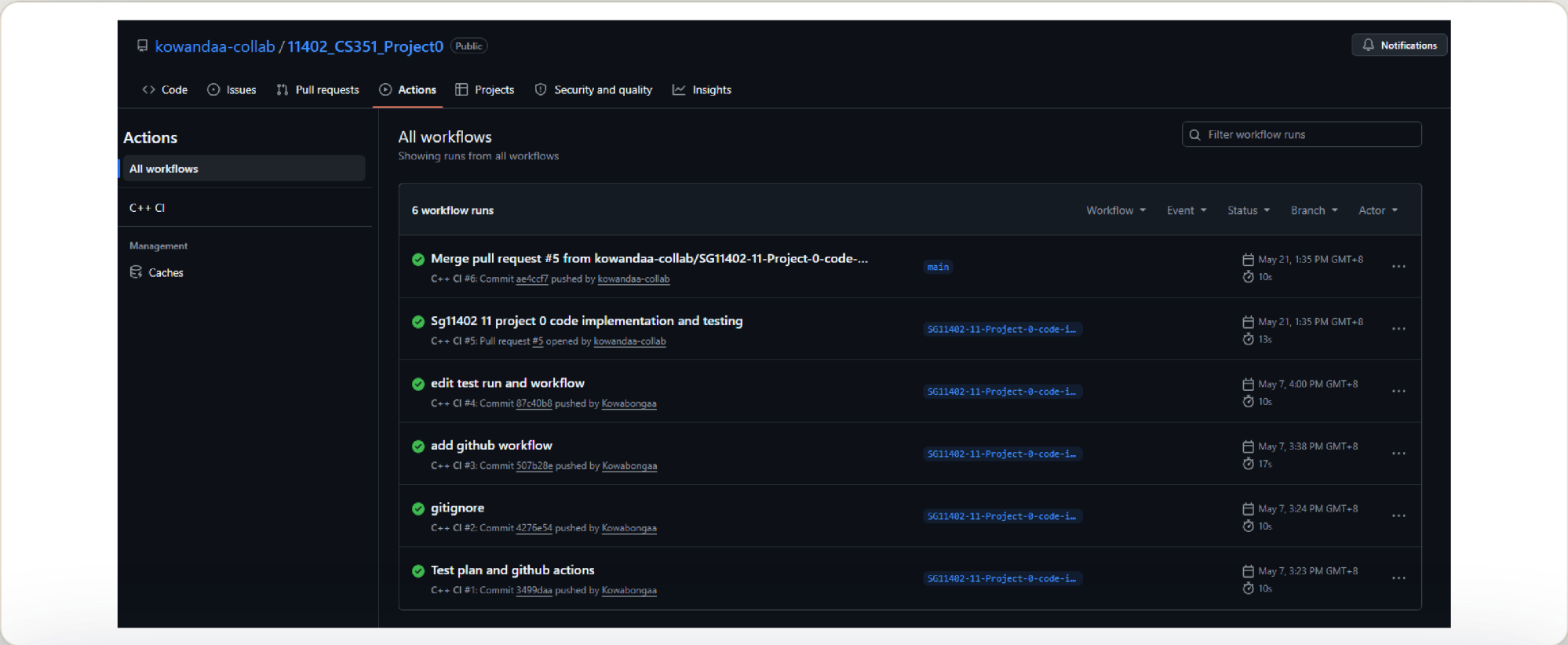
simple	{2, 7, 11, 15}, target 9
edge	negative & mixed signs
single	no solution → empty
stress	10,000-element array

✓ ALL TESTS PASSED `exit 0`

```
Kowabongaa Test plan and github actions ✓
Code Blame 88 lines (73 loc) • 2.79 KB
1 #include <iostream>
2 #include <string>
3 #include <vector>
4
5 std::vector<int> twosum(const std::vector<int>& nums, int target);
6 std::vector<int> twosumarray(const std::vector<int>& nums, int target);
7 std::vector<int> twosumhashable(const std::vector<int>& nums, int target);
8
9 static std::string vectostring(const std::vector<int>& v) {
10     std::string s = "[";
11     for (size_t i = 0; i < v.size(); ++i) {
12         s += std::to_string(v[i]);
13         if (i + 1 < v.size()) s += ", ";
14     }
15     s += "]";
16     return s;
17 }
18
19 static int runOne(const std::string& name,
20                  const std::vector<int>& nums,
21                  int target,
22                  const std::vector<int>& expected) {
23     const auto gotarrute = twosumarray(nums, target);
24     const auto gothash = twosumhashable(nums, target);
25     const auto gotreq = twosum(nums, target);
26
27     const bool okarrute = (gotarrute == expected);
28     const bool okhash = (gethash == expected);
29     const bool okreq = (gotreq == expected);
30
31     std::cout << name << "V\n";
32     std::cout << "  nums" << vectostring(nums) << " target" << target << "V\n";
33     std::cout << "  expected:" << vectostring(expected) << "V\n";
34     std::cout << "  twosumarray    " << vectostring(gotarrute) << (okarrute ? " PASS" : " FAIL") << "V\n";
35     std::cout << "  twosumhashable  " << vectostring(gothash) << (okhash ? " PASS" : " FAIL") << "V\n";
36     std::cout << "  twosum (required)" << vectostring(gotreq) << (okreq ? " PASS" : " FAIL") << "V\nV\n";
37
38     return (okarrute && okhash && okreq) ? 0 : 1;
39 }
40
41 int main() {
42     // User input mode:
43     // Format: n then n integers then target
44     // Example:
45     // 4
46     // 2 7 11 15
47     // 9
48     int n = 0;
49     if (std::cin >> n) {
50         std::vector<int> nums;
51         nums.reserve(n);
52         for (int i = 0; i < n; ++i) {
53             int x = 0;
54             if (!(std::cin >> x)) {
55                 std::cerr << "Invalid input: expected " << n << " integers.V\n";
56                 return 1;
57             }
58             nums.push_back(x);
59         }
60
61         int target = 0;
62         if (!(std::cin >> target)) {
63             std::cerr << "Invalid input: expected target integer.V\n";
64             return 1;
65         }
66
67         const auto ans = twosum(nums, target);
68         std::cout << vectostring(ans) << "V\n";
69         return 0;
70     }
71
72     // No input provided => run the ONLY test cases (per user request).
73     int failures = 0;
74     failures += runOne("simple case", {1, 2, 11, 15}, 9, {0, 1});
75     failures += runOne("edge case", {1, -2, -3, 5, 10}, 0, {1, 4});
76     failures += runOne("single amount", {0}, 10, {});
77
78     std::vector<int> big(10'000, 1);
79     big[123] = 100;
80     big[5076] = 200;
81     failures += runOne("stress case", big, 300, {123, 5076});
82
83     std::cout << (failures == 0 ? "ALL TESTS PASSEDV\n" : "SOME TESTS FAILEDV\n");
84     return failures == 0 ? 0 : 1;
85 }
```

Continuous Integration

• 6 / 6 runs passing



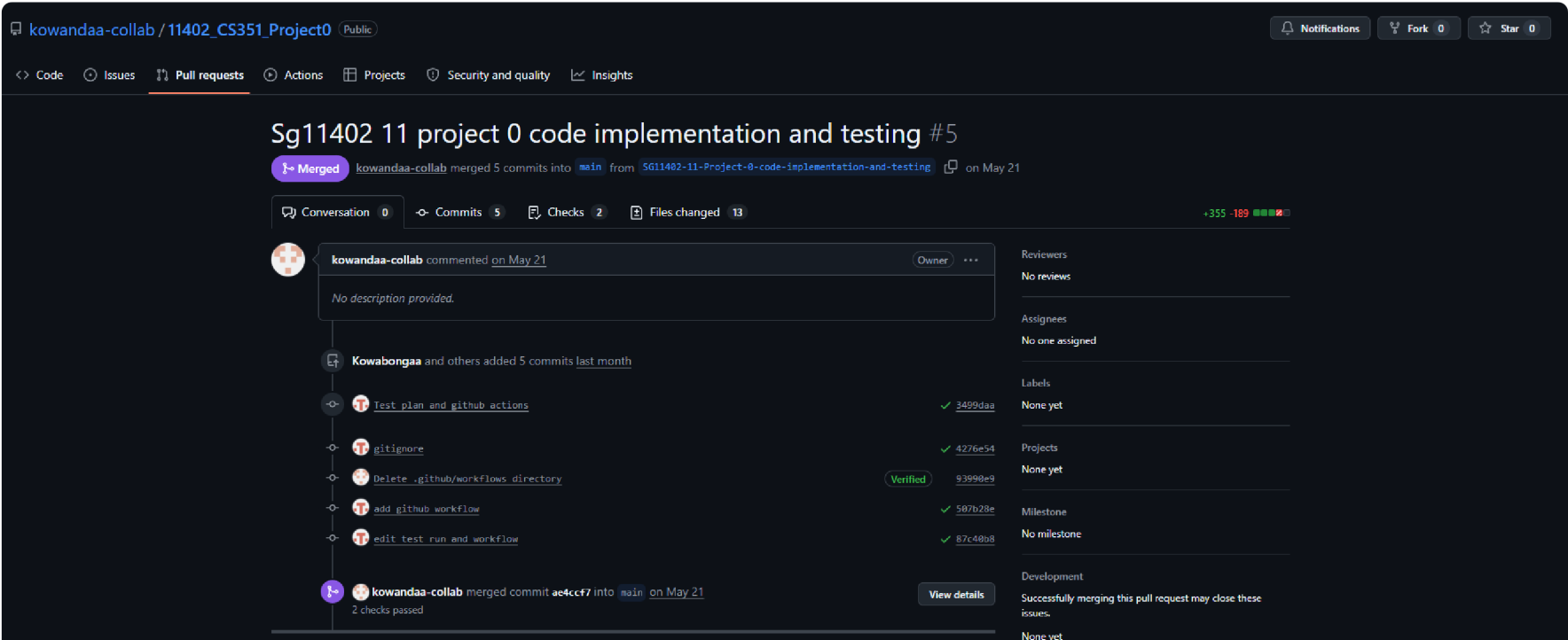
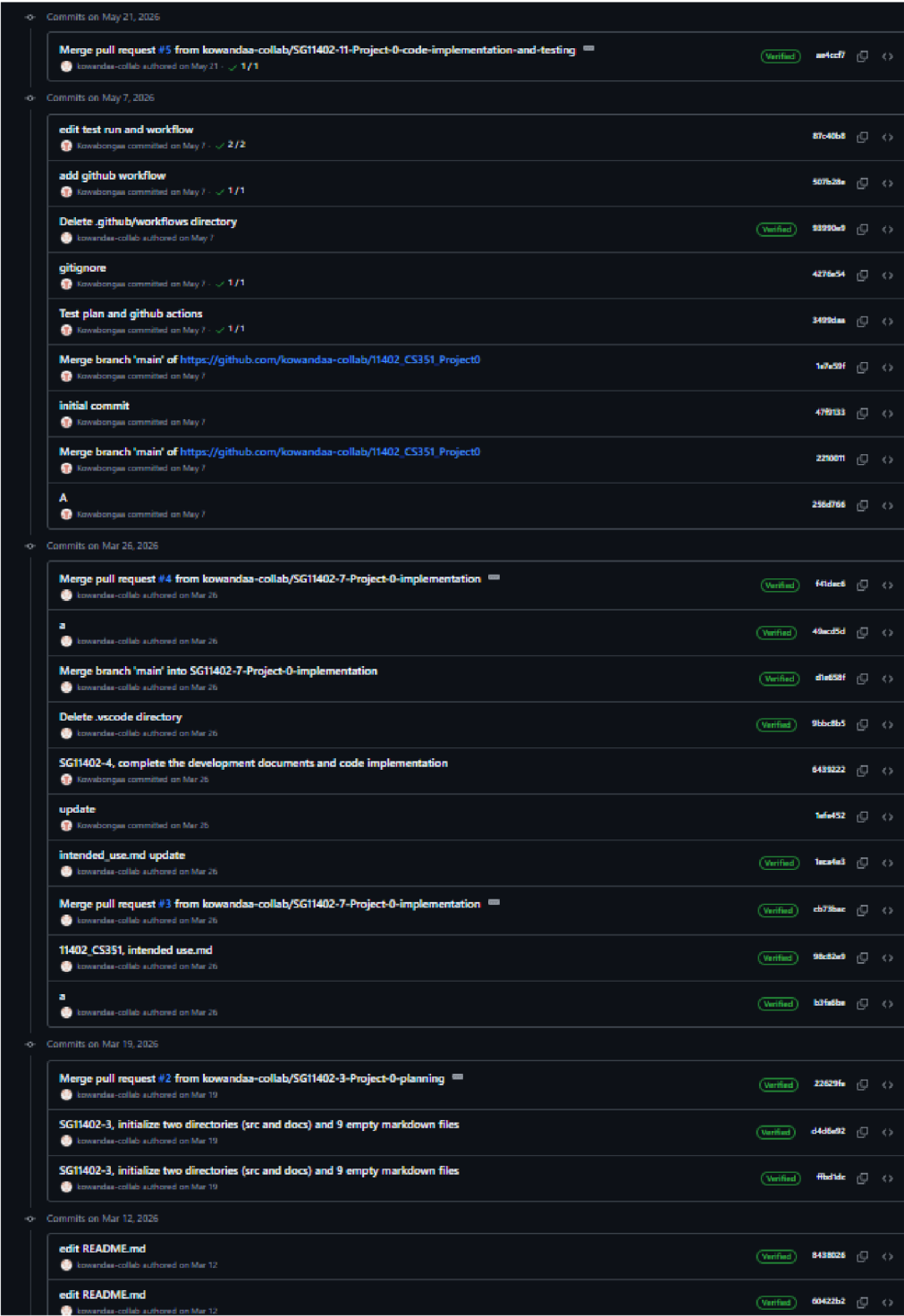
PIPELINE • CI.YML ON PUSH

- ✓ Checkout repository
- ✓ Build tests (g++)
- ✓ Run tests
- ✓ Complete job ~10s

Every push must pass before merge — automated proof.

CROSS-PROJECT · WORKFLOW MATURITY

GitHub Workflow & History



30
commits

5
pull requests

2
contributors

CSV Mini Database

A command-line database over CSV files — load, display, query and edit records.

`read` parse CSV into records

`query` find rows by input

`write` save changes back

Preview Code Blame 9 lines (9 loc) · 233 Bytes					
Q Search this file					
1	id	name	age	salary	department
2	1	Alice	30	50000	Engineering
3	2	Bob	25	45000	Sales
4	3	Charlie	35	60000	Engineering
5	4	Diana	28	48000	Sales
6	5	Eve	32	55000	Engineering
7	6	Frank	26	47000	Sales
8	7	Grace	31	52000	Engineering
9	8	Henry	29	49000	HR

Functions & Architecture

QUERY ENGINE OVER CSV

- SELECT project chosen columns
- WHERE filter rows by condition
- GROUP BY · COUNT aggregate per group
- ORDER BY sort ascending / descending

LANGUAGE

Python · CSV parsed into rows

MODULARITY

parse · query · I/O separated

TEST EVIDENCE

✓ 185 tests passed

pytest · 0.49s · 100%

```
PS C:\Users\alvin\11402_CS351_ProjectB> python -m pytest -q
..... [ 38%]
..... [ 77%]
..... [100%]
185 passed in 0.49s
PS C:\Users\alvin\11402_CS351_ProjectB> 
```

Querying the CSV

```
PS C:\Users\alvin\11402_CS351_ProjectB> python demo.py
>>> SELECT * FROM employees
-----
id  name  age  salary  department
--  -
1   Alice  30   50000   Engineering
2   Bob    25   45000   Sales
3   Charlie 35   60000   Engineering
4   Diana  28   48000   Sales
5   Eve    32   55000   Engineering
6   Frank  26   47000   Sales
7   Grace  31   52000   Engineering
8   Henry  29   49000   HR
(8 row(s))

>>> SELECT name, department, salary FROM employees WHERE salary > 50000
-----
name  department  salary
-----
Charlie Engineering 60000
Eve    Engineering 55000
Grace  Engineering 52000
(3 row(s))

>>> SELECT department, COUNT(*) FROM employees GROUP BY department
-----
department  COUNT
-----
Engineering  4
Sales        3
HR           1
(3 row(s))

>>> SELECT name, age FROM employees ORDER BY age DESC
-----
name  age
-----
Charlie 35
Eve    32
Grace  31
Alice  30
Henry  29
Diana  28
Frank  26
Bob    25
(8 row(s))

>>> SELECT name, salary FROM employees WHERE department = 'Engineering' ORDER BY salary DESC
-----
name  salary
-----
Charlie 60000
Eve    55000
Grace  52000
Alice  50000
```

SELECT * FROM employees
Loads & prints all **8 rows** as a table.

WHERE salary > 50000
Filters down to **3 rows**.

GROUP BY department
Counts **Eng 4 • Sales 3 • HR 1**.

ORDER BY age DESC
Sorts oldest → youngest.

Prompt, Verify, Judge

01 — Prompting

- › Asked `unordered_map` approach for the
- › Wrote the constraints in my own words

I framed it — AI drafted it.

02 — Verification

- › Read the code, checked the no-reuse rule
- › Let CI run all four test cases

Green CI, not AI confidence.

03 — Human Judgement

- › Chose structure, branches, what to merge
- › Failing test > the chat as truth

I owned every merge to main.

Reflection & Future Work

WHAT I LEARNED

- 01 Git workflow makes teamwork safe.
- 02 Tests make correctness provable.
- 03 AI drafts; humans verify.

IF I HAD MORE TIME

- Persistent storage for Project B
- A richer query language
- A small web front end
- Wider test coverage

CS351 · FINAL DEMO

Thank you.

Questions welcome — algorithm, tests, Git workflow, or AI process.

11402_CS351_Project0

11402_CS351_ProjectB

github.com/kowandaa-collab